

## **TEST PLAN – SOFTWARE TESTING**

Software testing is important to make sure applications work properly and meet user needs. A clear and detailed test plan is the foundation of successful testing, guiding everything from creating test cases to fixing issues.

### **What is a Test Plan?**

A test plan is a document that consists of all future testing-related activities. It is prepared at the project level and in general, it defines work products to be tested, how they will be tested, and test type distribution among the testers. Before starting testing there will be a test manager who will be preparing a test plan.

The test plan serves as the blueprint that changes according to the progressions in the project and stays current at all times.

It serves as a base for conducting testing activities and coordinating activities among a QA team.

It is shared with Business Analysts, Project Managers, and anyone associated with the project.

### **Types of Test Plan**

#### **1. Master Test Plan**

A Master Test Plan is a high-level document that outlines the overall testing strategy, scope, and phases for a project. It provides a roadmap for the entire testing process and shows how different levels of testing connect with each other. This plan ensures that all testing activities are aligned and helps keep the project on track.

#### **2. Specific Test Plan**

A Specific Test Plan focuses on particular types of testing, like load testing, performance testing, or security testing. It provides detailed instructions and guidelines for carrying out these specialized tests, ensuring that each area is tested thoroughly.

#### **3. Analytical Test Strategies**

Analytical Test Strategies are based on specific factors like project requirements, specifications, or risks. For example:

User Acceptance Testing is based on user requirements to make sure the software meets the needs of the end-users.

System Testing is based on the technical specifications to check if each part of the system functions correctly.

### **How to create a Test Plan?**

#### **1. Analyze the product**

This phase focuses on analyzing the product, Interviewing clients, designers, and developers, and performing a product walkthrough. This stage focuses on answering the following questions:

What is the primary objective of the product?

Who will use the product?

What are the hardware and software specifications of the product?

How does the product work?

#### **2. Design the test strategy**

The test strategy document is prepared by the manager and details the following information:

Scope of testing which means the components that will be tested and the ones that will be skipped.

Type of testing which means different types of tests that will be used in the project.

Risks and issues that will list all the possible risks that may occur during testing.

Test logistics mentions the names of the testers and the tests that will be run by them.

### 3. Define test objectives

This phase defines the objectives and expected results of the test execution.

Objectives include:

A list of software features like functionality, GUI, performance standards, etc.

The ideal expected outcome for every aspect of the software that needs testing.

### 4. Define test criteria

Two main testing criteria determine all the activities in the testing project:

**Suspension criteria:** Suspension criteria define the benchmarks for suspending all the tests.

**Exit criteria:** Exit criteria define the benchmarks that signify the successful completion of the test phase or project. These are expected results and must match before moving to the next stage of development.

### 5. Resource planning

This phase aims to create a detailed list of all the resources required for project completion. For example, human effort, hardware and software requirements, all infrastructure needed, etc.

### 6. Plan test environment

This phase is very important as the test environment is where the QAs run their tests. The test environments must be real devices, installed with real browsers and operating systems so that testers can monitor software behavior in real user conditions.

### 7. Schedule and Estimation

Break down the project into smaller tasks and allocate time and effort for each task. This helps in efficient time estimation. Create a schedule to complete these tasks in the designated time with a specific amount of effort.

### 8. Determine test deliverables

Test deliverables refer to the list of documents, tools, and other equipment that must be created, provided, and maintained to support testing activities in the project.

#### Why is Test Plan creation important?

**Defines Objectives:** A test plan clearly outlines the testing objectives and the scope of testing activities, ensuring that all team members understand what needs to be achieved.

**Structured Approach :** It provides a systematic approach to testing, detailing the steps and processes involved, which helps in organizing the testing effort.

**Avoids Scope Creep :** By defining what will and will not be tested, the test plan helps manage the scope of testing activities, preventing unnecessary work and focusing on irrelevant areas.

**Resource Allocation :** Helps in identifying the necessary resources, including personnel, tools, and environments, ensuring they are available when needed.

**Identifies Risks :** A test plan identifies potential risks and outlines mitigation strategies, helping to address issues proactively rather than reactively.

**Contingency Plans :** These include contingency plans for dealing with unexpected events or issues that may arise during testing.

**Stakeholder Alignment :** Facilitates communication among stakeholders, including developers, testers, project managers, and clients, ensuring everyone is aligned on the testing objectives, approach, and schedule.

**Documentation :** Serves as a comprehensive document that can be referred to by all team members, aiding in knowledge sharing and transparency.

**Resource Optimization :** Helps in efficiently utilizing available resources, including time and personnel, by providing a clear plan of action.

**Focus on Priorities :** Ensures that testing efforts are focused on high-priority

areas that are critical to the success of the project.

### **Objectives of the Test Plan**

**Overview of testing activities:** The test plan provides an overview of the testing activities and where to start and stop the work.

**Provides timeline:** The test plan helps to create the timeline for the testing activities based on the number of hours and the workers needed.

**Helps to estimate resources:** The test plan helps to create an estimate of the number of resources needed to finish the work.

**Serves as a blueprint:** The test plan serves as a blueprint for all the testing activities, it has every detail from beginning to end.

**Helps to identify solutions:** A test plan helps the team members. They consider the project's challenges and identify the solutions.

**Serves as a rulebook:** The test plan serves as a rulebook for following rules when the project is completed phase by phase.

### **Components and Attributes of Test Plan**

1. **Objective:** It describes the aim of the test plan, whatever the good process and procedure they are going to follow to give quality software to customers. The overall objective of the test is to find as many defects as possible and to make software bug-free. The test objective must be broken into components and sub-components. In every component following activities should be performed.

List all the functionality and performance to be tested.

Make goals and targets based on the application feature.

2. **Scope:** It consists of information that needs to be tested concerning an application. The scope can be divided into two parts:

**In-Scope:** The modules that are to be tested rigorously.

**Out Scope:** The modules that are not to be tested rigorously.

**Example:** In an application A, B, C, and D features have to be developed, but the B feature has already been designed by other companies. So the development team will purchase B from that company and perform only integrated testing with A, B, and C.

3. **Testing Methodology:** The methods that are going to be used for testing depend on application to application. The testing methodology is decided based on the feature and application requirements. Since the testing terms are not standard, one should define what kind of testing will be used in the testing methodology. So that everyone can understand it.

4. **Approach:** The approach of testing different software is different. It deals with the flow of applications for future reference. It has two aspects:

**High-Level Scenarios:** For testing critical features high-level scenarios are written. For Example, login to a website, and book from a website.

**The Flow Graph:** It is used when one wants to make benefits such as converging and merging easy.

5. **Assumption:** In this phase, certain assumptions will be made.

**Example:**

The testing team will get proper support from the development team.

The tester will get proper knowledge transfer from the development team.

Proper resource allocation will be given by the company to the testing department.

6. **Risk:** All the risks that can happen if the assumption is broken. For Example, in the case of wrong budget estimation, the cost may overrun. Some reason that may lead to risk is:

Test Manager has poor management skills.

Hard to complete the project on time.

Lack of cooperation.

7. **Mitigation Plan:** If any risk is involved then the company must have a backup plan, the purpose is to avoid errors. Some points to resolve/avoid risk:

Test priority is to be set for each test activity.

Managers should have leadership skills.

Training course for the testers.

8. Roles and Responsibilities: All the responsibilities and role of every member of a particular testing team has to be recorded.

Example:

Test Manager: Manages the project, takes appropriate resources, and gives project direction.

Tester: Identify the testing technique, verify the test approach, and save project costs.

9. Schedule: Under this, it will record the start and end date of every testing-related activity. For Example, writing the test case date and ending the test case date.

10. Defect Tracking: It is an important process in software engineering as lots of issue arises when you develop a critical system for business. If there is any defect found while testing that defect must be given to the developer team. There are the following methods for the process of defect tracking:

Information Capture: In this, we take basic information to begin the process.

Prioritize: The task is prioritized based on severity and importance.

Communication: Communication between the identifier of the bug and the fixer of the bug.

Environment: Test the application based on hardware and software.

Example: The bug can be identified using bug-tracking tools such as Jira, Mantis, and Trac.

11. Test Environments: It is the environment that the testing team will use i.e. the list of hardware and software, while testing the application, the things that are said to be tested will be written under this section. The installation of software is also checked under this.

Example:

Software configuration on different operating systems, such as Windows, Linux, Mac, etc.

Hardware Configuration depends on RAM, ROM, etc.

12. Entry and Exit Criteria: The set of conditions that should be met to start any new type of testing or to end any kind of testing.

Entry Condition:

Necessary resources must be ready.

The application must be prepared.

Test data should be ready.

Exit Condition:

There should not be any major bugs.

Most test cases should be passed.

When all test cases are executed.

13. Test Automation: It consists of the features that are to be automated and which features are not to be automated.

If the feature has lots of bugs then it is categorized as Manual Testing.

If the feature is frequently tested then it can be automated.

14. Effort Estimation: This involves planning the effort that needs to be applied by every team member.

15. Test Deliverables: It is the outcome from the testing team that is to be given to the customers at the end of the project.

### **Creating an effective Test Plan**

Creating an effective test plan is essential for ensuring a comprehensive and systematic approach to software testing. Here are some best practices to consider

when developing a test plan:

1. Understand the Project Requirements:

**Gather Information:** Ensure a thorough understanding of both functional and non-functional requirements.

**Stakeholder Input:** Involve stakeholders to gather expectations and address specific concerns.

2. Define Clear Objectives and Scope:

**Purpose of Testing:** Clearly state the objectives and what you aim to achieve.

**In-Scope and Out-of-Scope:** Define what will be tested and what will not, to manage expectations and focus resources.

3. Develop a Comprehensive Test Strategy:

**Approach:** Outline the types of testing to be performed (e.g., functional, regression, performance).

**Techniques and Tools:** Specify testing techniques (e.g., black-box, white-box) and tools (e.g., Selenium, JIRA) to be used.

4. Create Detailed Test Cases:

**Test Case Design:** Develop detailed test cases covering all scenarios, including positive, negative, edge, and boundary cases.

**Traceability:** Ensure each test case is traceable to specific requirements to verify comprehensive coverage.

5. Establish a Test Environment:

**Setup Requirements:** Define hardware, software, network configurations, and tools required for testing.

**Environment Management:** Ensure the test environment closely mirrors the production environment to identify environment-specific issues.

6. Plan for Test Data and Reporting Mechanisms:

**Data Requirements:** Identify and manage realistic, consistent test data securely, especially if it includes sensitive information.

**Status Reporting:** Establish processes for regular status updates on testing progress, issues, and results, and use defect tracking systems effectively.

## TEST STRATEGY

**Test strategy** is a document that which the **Testing** of any application, which is organized in a well-documented way with proper goals.

**Test strategy** is a plan for defining an approach to the Software Testing Life Cycle (STLC).

In addition, the test strategy provides the following details, which are required while writing the test document:

What technique must be used in addition to this?

Which of the modules will be examined?

What criteria apply for entry and exit?

What kind of testing is necessary?

### Components of a Test Strategy

- The test effort, test domain, test setups, and test tools used to verify and validate a set of functions are all outlined in a Test Strategy.
- It also includes schedules, resource allocations, and employee utilization information. This data is essential for the test team (Test) to be as structured and efficient as possible.
- A Test Strategy differs from a Test Plan, which is a document that gathers and organizes test cases by functional areas and/or types of testing in a format that can be presented to other teams and/or customers.
- Both are critical components of the Quality Assurance process since they aid in communicating the breadth of the test method and ensuring test coverage while increasing the testing effort's efficiency.

Below diagram shows the components of the test strategy:



#### 1. Scope and Overview

An overview of the project, as well as information on who should utilize this page.

Include information such as who will evaluate and approve the document.

Define the testing activities and phases that will be performed, as well as the timetables that will be followed in relation to the overall project timelines stated in the test plan.

#### 2. Testing Methodology

Testing methodology is used to specify the degrees of testing, testing procedures, roles, and duties of all team members. The change management process, which includes the modification request submission, pattern to be utilized, and activity to manage the request, is also included in the testing strategy.

Define the testing process, testing level, roles, and duties of each team member.

Describe why each test type is defined in the test plan (for example, unit, integration, system, regression, installation/uninstallation, usability, load, performance, and security testing) should be performed.

#### 3. Testing Environment Specifications

The testing environment specification in the test strategy document includes clear instructions on how to produce test data.

The information about the number of environments and the needed configuration for each environment should be included in the test environment setup.

For example, the functional test team might have one test environment and the UAT team might have another.

Define the number of users supported in each environment, as well as each user's access roles and software and hardware requirements, such as the operating system, RAM, free disc space, and the number of systems.

It's just as crucial to define the test data needs.

Give specific instructions on how to generate test data (either generate data or use production data by masking fields for privacy).

Define a backup and restoration strategy for test data.

Due to unhandled circumstances in the code, the test environment database may encounter issues.

The backup and restoration method should state who will take backups when backups should be taken, what should be included in backups, when the database should be restored, who will restore it, and what data masking procedures should be implemented if the database is restored.

#### 4. Testing Tools

Testing tools are an important part of the test strategy document since it contains all of the information on the test management and automation tools that are required for test execution.

Define the tools for test management and automation that will be utilized to execute the tests.

Describe the test approach and tools needed for performance, load, and security testing.

Mention whether the product is open-source or commercial, as well as the number of individuals it can accommodate, and make suitable planning.

#### 5. Release Control

Release Control is a crucial component of the test strategy document. It's used to make sure that test execution and release management strategies are established in a systematic way. It specifies the following information-

Different software versions in test and UAT environments can occur from unplanned release cycles.

All adjustments in that release will be tested using the release management strategy, which includes a proper version history.

Set up a build management process that answers questions like where the new build should be made available, where it should be deployed when to receive the new build, where to acquire the production build, who will give the go signal for the production release, and so on.

#### 6. Risk Analysis

Risk Analysis is the next section of the test strategy paper. All potential hazards associated with the project are described in the test strategy document and can become an issue during test execution.

#### 7. Review and Approval

Review and Approval is the last section of the Testing strategy paper.

When all of the testing activities are stated in the test strategy document, it is evaluated by the persons that are involved, such as:

**System Administration Team.**

**Project Management Team.**

**Development Team.**

**Business Team.**

### RESOURCE REQUIREMENTS

- Resource requirement is a detailed summary of all types of resources required to complete project task.
- Resource could be human, equipment and materials needed to complete a project.
- The resource requirement and planning is important factor of the test planning because helps in determining the number of resources (employee, equipment...) to be used for the project.
- Therefore, the Test Manager can make the correct schedule & estimation for the project.

Some of the following factors need to be considered:

- Machine configuration (RAM,processor,disk)needed to run the product under test.
- Overheads required by test automation tools, if any
- Supporting tools such as compilers, test data generators, configuration management tools.
- The different configurations of the supporting software(e.g. OS)that must be present
- Special requirements for running machine-intensive tests such as load tests and performance tests.
- Appropriate number of licenses of all the software

### Human Resource & System Resource:

For testing, a web application, you should plan the resources.

#### 1 Server

Install the web application under test. This includes a separate web server, database server, and application server if applicable

#### 2 Test tool

The testing tool is to automate the testing, simulate the user operation, and generate the test results. There are tons of test tools you can use for this project such as Selenium, QTP...etc.

#### 3. Network

You need a Network include LAN and Internet to simulate the real business and user environment.

### TESTER ASSIGNMENTS

- They may be involved in or even be the primary people identifying test conditions and creating test designs, test cases, test procedure specifications and test data, and may automate or help to automate the tests.
- Tester assignments typically refer to the process of assigning individuals or teams to test specific components, features, or aspects of a product or system.
- Testing is an essential part of software development, quality assurance, and product validation. Assigning testers to different tasks helps ensure thorough and efficient testing coverage.

Here are some considerations for tester assignments:

**Test plan and strategy:** Begin by creating a test plan and strategy that outlines the testing objectives, scope and approach. This will help identify the different types of tests required and the specific areas that need to be covered.

**Test case creation:** Testers can be assigned to create test cases based on the test plan and requirements. Test cases should cover different scenarios and use cases to ensure comprehensive testing.

**Testing types:** Identify the different types of testing needed, such as functional testing, performance testing, security testing, usability testing, etc. Assign testers with the relevant expertise and skills for each type of testing.

**Test environment:** Assign testers to set up and configure the test environment, including any necessary hardware, software, or network configurations. This ensures that the testing environment accurately reflects the production environment.

**Test execution:** Assign testers to execute the test cases and document the results. Testers should follow the test plan and report any issues or bugs they encounter during the testing process.

**Bug reporting and tracking:** Assign testers to report identified bugs or issues in a structured manner, including detailed descriptions, steps to reproduce, and any supporting documentation. Testers may also be responsible for tracking the status and resolution of reported issues.

**Regression testing:** After bug fixes or changes are made, assign testers to perform regression testing to ensure that the fixes or changes did not introduce new issues or break existing functionality.

**Collaboration and communication:** Assign testers to collaborate with developers, product managers, and other stakeholders to ensure clear communication and understanding of testing requirements, priorities, and progress.

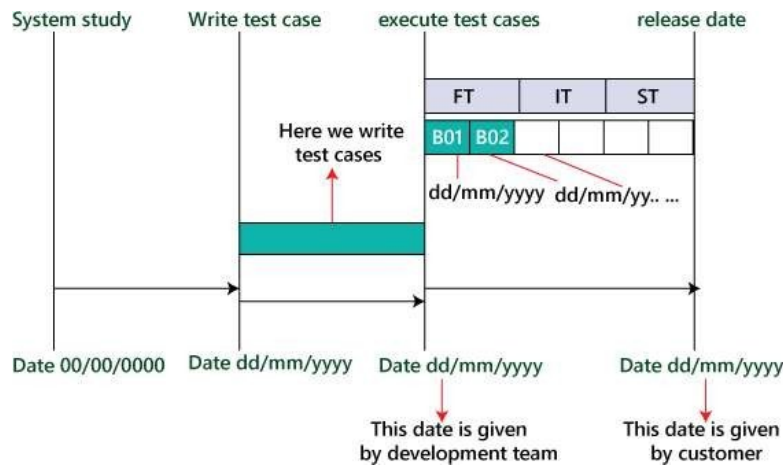
**Test coverage analysis:** Assign testers to analyze the test coverage and identify any gaps or areas that require additional testing. This helps ensure that all critical features and functionalities are adequately tested.

**Test automation:** Assign testers with automation skills to develop and maintain automated test scripts, which can help increase testing efficiency and coverage.

#### Test Schedule:

It is used to explain the timing to work, which needs to be done or this attribute covers when exactly each testing activity should start and end? And the exact data is also mentioned for every testing activity for the particular date.





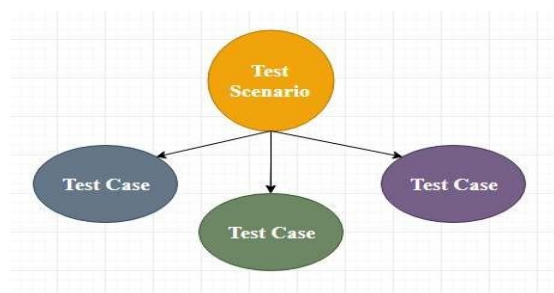
Therefore, as we can see in the below image that for the particular activity, there will be a starting date and ending date; for each testing to a specific build, there will be the specified date.

For example

Writing test cases

Execution process

## TEST CASES



The test case is defined as a group of conditions under which a tester determines whether a software application is working as per the customer's requirements or not. Test case designing includes preconditions, case name, input conditions, and expected result. A test case is a first level action and derived from test scenarios.

It is an in-details document that contains all possible inputs (positive as well as negative) and the navigation steps, which are used for the test execution process. Writing of test cases is a one-time attempt that can be used in the future at the time of regression testing.

Test case gives detailed information about testing strategy, testing process, preconditions, and expected output. These are executed during the testing process to check whether the software application is performing the task for that it was developed or not.

Test case helps the tester in defect reporting by linking defect with test case ID.

Detailed test case documentation works as a full proof guard for the testing team because if developer missed something, then it can be caught during execution of these full-proof test cases.

To write the test case, we must have the requirements to derive the inputs, and the test scenarios must be written so that we do not miss out on any features for testing. Then we should have the test case template to maintain the uniformity, or every test engineer follows the same approach to prepare the test document.

Generally, we will write the test case whenever the developer is busy in writing the code.

### When do we write a test case?

We will write the test case when we get the following:

When the customer gives the business needs then, the developer starts developing and says that they need 3.5 months to build this product.

And In the meantime, the testing team will start writing the test cases.

Once it is done, it will send it to the Test Lead for the review process.

And when the developers finish developing the product, it is handed over to the testing team.

The test engineers never look at the requirement while testing the product document because testing is constant and does not depend on the mood of the person rather than the quality of the test engineer.

### **Why we write the test cases?**

We will write the test for the following reasons:

- To require consistency in the test case execution
- To make sure a better test coverage
  
- It depends on the process rather than on a person
- To avoid training for every new test engineer on the product
- To require consistency in the test case execution: we will see the test case and start testing the application.
- To make sure a better test coverage: for this, we should cover all possible scenarios and document it, so that we need not remember all the scenarios again and again.
- It depends on the process rather than on a person: A test engineer has tested an application during the first release, second release, and left the company at the time of third release.
- As the test engineer understood a module and tested the application thoroughly by deriving many values.
- If the person is not there for the third release, it becomes difficult for the new person. Hence all the derived values are documented so that it can be used in the future.
- To avoid giving training for every new test engineer on the product: When the test engineer leaves, he/she leaves with a lot of knowledge and scenarios.
- Those scenarios should be documented so that the new test engineer can test with the given scenarios and also can write the new scenarios.

### **Test case template**

## Header

Test Case Name/ID :- **Release - Version - Application Name - Module**

Test Case Type:-

|       |       |       |
|-------|-------|-------|
| F.T.C | I.T.C | S.T.C |
|-------|-------|-------|

Requirement Number:-

Module:-

Severity:- **Critical/Major/Minor**

Status:-

Release:-

Version:-

Pre-condition:-

Test Data:-

Summary:-

## Body

| Step No. | Description | Inputs | Expected Result | Actual Result | Status | Comments |
|----------|-------------|--------|-----------------|---------------|--------|----------|
| ...      | ...         | ...    | ...             | ...           | ...    | ...      |
| ...      | ...         | ...    | ...             | ...           | ...    | ...      |

## Footer

Author:-

Reviewed By:-

Date:-

Approved By:-

The primary purpose of writing a test case is to achieve the efficiency of the application.

As we know, the actual result is written after the test case execution, and most of the time, it would be same as the expected result. But if the test step will fail, it will be different. So, the actual result field can be skipped, and in the Comments section, we can write about the bugs.

And also, the Input field can be removed, and this information can be added to the description field.

The above template we discuss above is not the standard one because it can be different for each company and also with each application, which is based on the test engineer and the test lead. But for testing one application, all the test engineers should follow a usual template, which is formulated.

The test case should be written in simple language so that a new test engineer can also understand and execute the same. In the above sample template, the header contains the following:

Step number

It is also essential because if step number 20 is failing, we can document the bug report and hence prioritize working and also decide if it's a critical bug.

Test case type:

It can be functional, integration or system test cases or positive or negative or positive and negative test cases. Release:

One release can contain many versions of the release.

Pre-condition:

These are the necessary conditions that need to be satisfied by every test engineer before starting the test execution process. Or it is the data configuration or the data setup that needs to be created for the testing.

For example: In an application, we are writing test cases to add users, edit users, and delete users. The per-condition will be seen if user A is added before editing it and removing it.

Test data

These are the values or the input we need to create as per the per-condition.

For example, Username, Password, and account number of the users.

The test lead may be given the test data like username or password to test the application, or the test engineer may themselves generate the username and password.

Severity

The severity can be major, minor, and critical, the severity in the test case talks about the importance of that particular test cases. All the test execution process always depends on the severity of the test cases.

**Brief description**

The test engineer has written a test case for a particular feature.

If he/she comes and reads the test cases for the moment, he/she will not know for what feature has written it. So, the brief description will help them in which feature test case is written.

**Example of a test case template.**

Here, we are writing a test case for the ICICI application's Login module:

| Test case template |                                            |                  |                                 |                            |        |          |
|--------------------|--------------------------------------------|------------------|---------------------------------|----------------------------|--------|----------|
| test case name     | Delta-3 0-ICICI-Login                      |                  |                                 |                            |        |          |
| test case type     | Functional test case                       |                  |                                 |                            |        |          |
| requirement no     | 1                                          |                  |                                 |                            |        |          |
| module             | login                                      |                  |                                 |                            |        |          |
| status             | XXX                                        |                  |                                 |                            |        |          |
| severity           | critical                                   |                  |                                 |                            |        |          |
| release            | Delta                                      |                  |                                 |                            |        |          |
| version            | 3                                          |                  |                                 |                            |        |          |
| pre-condition      | required one login                         |                  |                                 |                            |        |          |
| test data          | username-abc, password-123                 |                  |                                 |                            |        |          |
| summary            | to check the functionality of login        |                  |                                 |                            |        |          |
| Steps-no           | Description                                | Inputs           | Expected result                 | Actual results             | Status | Comments |
| 1                  | open "Browser" and enter the "Url"         | https://QA/Main/ | Login page must be display      | As Expected                | pass   | XXX      |
| 2                  | enter the following values for "Username": |                  |                                 |                            |        |          |
|                    | Valid(abc)                                 | abc              | Accept                          | Login page must be display | pass   | XXX      |
|                    | Invalid                                    | 555              | Error message "invalid login"   | not as expected            | fail   | bug #1   |
|                    | Blank                                      | Null             | Error message username cannot t | not as expected            | fail   |          |
|                    | Symbols                                    | 2 alphabet       | Error message invalid login     | not as expected            | fail   |          |
| 3                  | enter the following values for "Password": |                  |                                 |                            |        |          |
|                    | valid                                      | 123              | Accept                          | Login page must be display | pass   | XXX      |
|                    | invalid                                    | xy3              | Error message invalid login     | not as expected            | fail   |          |
|                    | Blank                                      |                  | Error message password cannot t | not as expected            | fail   |          |
|                    | enter the valid username and password and  |                  |                                 |                            |        |          |
|                    | click on "OK" button                       | abc,123          | "home Pag" must be displayed    | home page is displayin     | pass   |          |
|                    | enter the valid username and password and  |                  |                                 |                            |        |          |
|                    | click on "Cancel" button                   | abc,123          | all field must be cleard        | the entered data is clea   | pass   | XXX      |
| author             | test engineer name                         |                  |                                 |                            |        |          |
| date               | 1/4/2020                                   |                  |                                 |                            |        |          |
| reviewed by        | ryan                                       |                  |                                 |                            |        |          |
| approved by        | jessica                                    |                  |                                 |                            |        |          |

## TYPES OF TEST CASES

We have a different kind of test cases, which are as follows:

Function test cases

Integration test cases

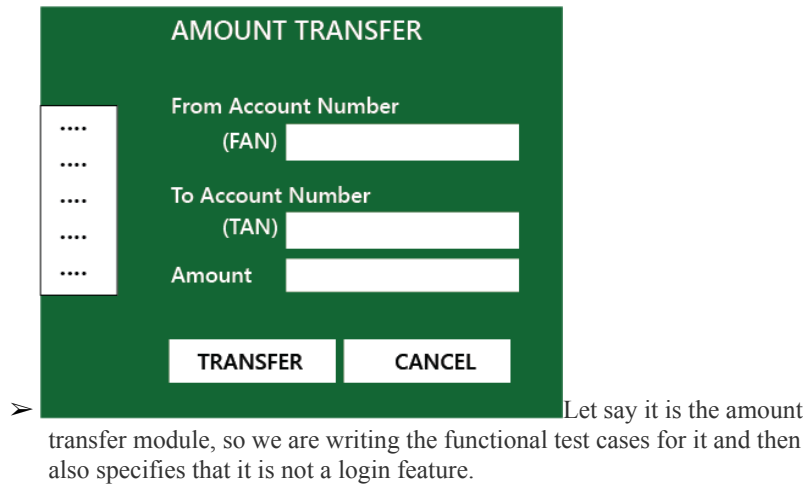
System test cases

## FUNCTIONAL TEST CASES

Firstly, we check for which field we will write test cases and then describe accordingly. In functional testing or if the application is data-driven, we require the input column else; it is a bit time-consuming.

Rules to write functional test cases:

- In the expected results column, try to use should be or must be.
- Highlight the Object names.
- We have to describe only those steps which we required the most; otherwise, we do not need to define all the steps.
- To reduce the excess execution time, we will write steps correctly.
- Write a generic test case; do not try to hard code it.



The functional test case for amount transfer module is in the below Excel file:

| Functional Test case template |                                                                                       |                      |                                                                       |                |        |          |
|-------------------------------|---------------------------------------------------------------------------------------|----------------------|-----------------------------------------------------------------------|----------------|--------|----------|
| Test case name                | beta-1.0-ICIQ-amount transfer                                                         |                      |                                                                       |                |        |          |
| Test case type                | Functional test case                                                                  |                      |                                                                       |                |        |          |
| Requirement no                | 6                                                                                     |                      |                                                                       |                |        |          |
| Module                        | amount transfer                                                                       |                      |                                                                       |                |        |          |
| Status                        | XXX                                                                                   |                      |                                                                       |                |        |          |
| Severity                      | Critical                                                                              |                      |                                                                       |                |        |          |
| Release                       | Beta                                                                                  |                      |                                                                       |                |        |          |
| Version                       | 1                                                                                     |                      |                                                                       |                |        |          |
| Pre-condition                 | sender login<br>two account number<br>Balance-> exist                                 |                      |                                                                       |                |        |          |
| Test data                     | User name:xyz, Password:1234<br>1231, 4321<br>3000-9000                               |                      |                                                                       |                |        |          |
| Summary                       | to check the functionality of amount transfer                                         |                      |                                                                       |                |        |          |
| Steps no                      | Description                                                                           | Inputs               | Expected result                                                       | Actual results | Status | Comments |
| 1                             | Open "Browser" and enter the "Url"                                                    | https://QA/Main/     | "Login page" must be display                                          | As Expected    | pass   | XXX      |
| 2                             | Enter the following values for "Username" and "Password" and click on the "OK" button | xyz, 1234            | "Home page" must be displayed                                         | As Expected    | pass   | XXX      |
| 3                             | Click on the "Amount Transfer"                                                        | Null                 |                                                                       |                | pass   | XXX      |
| 4                             | Enter the following for From Account number (FAN):                                    |                      |                                                                       |                |        |          |
|                               | valid                                                                                 | 1234                 | Accept                                                                | As Expected    | pass   |          |
|                               | invalid                                                                               | 1124                 | Error message invalid account                                         |                | fail   |          |
|                               | blank                                                                                 |                      | Error message FAN value cannot be blank                               |                | fail   |          |
|                               |                                                                                       |                      |                                                                       |                |        |          |
|                               |                                                                                       |                      | test maximum coverage                                                 |                |        |          |
| 5                             | Enter the following values for "TO account number" (TAN)                              |                      |                                                                       |                |        |          |
|                               | valid                                                                                 | 4321                 | Accept                                                                | As expected    | pass   | XXX      |
|                               | invalid                                                                               | 6655                 | Error message invalid account                                         |                | fail   |          |
|                               | Blank                                                                                 |                      | Error message TAN value cannot be blank                               |                |        |          |
|                               |                                                                                       |                      |                                                                       |                |        |          |
|                               |                                                                                       |                      | test maximum coverage                                                 |                |        |          |
| 6                             | enter the value for "Amount"                                                          |                      |                                                                       |                |        |          |
|                               | valid                                                                                 | 5000, 5001, 9000, 84 | Accept                                                                | As expected    | pass   | XXX      |
|                               | invalid                                                                               | 4999, 9001           | error message amount should be between (5000-9000)                    |                | fail   |          |
| 7                             | Enter the value for "FAN, TAN, Amount" click on the "Transfer" button                 |                      |                                                                       |                |        |          |
|                               | FAN                                                                                   | 1234                 |                                                                       |                |        |          |
|                               | TAN                                                                                   | 4321                 | "Confirmation Message" amount transfer successfully must be displayed | As expected    | pass   | XXX      |
|                               | Amount                                                                                | 6000                 |                                                                       |                |        |          |
| 8                             | Enter the value for "FAN, TAN, Amount" click on the "Cancel" button                   |                      |                                                                       |                |        |          |
|                               | FAN                                                                                   | 1234                 |                                                                       |                |        |          |
|                               | TAN                                                                                   | 4321                 | All field must be cleared                                             | As expected    | pass   | XXX      |
|                               | Amount                                                                                | 6000                 |                                                                       |                |        |          |
| Author                        | Sera                                                                                  |                      |                                                                       |                |        |          |
| Date                          | 4/1/2020                                                                              |                      |                                                                       |                |        |          |
| Reviewed by                   | jessica                                                                               |                      |                                                                       |                |        |          |
| Approved by                   | ryan                                                                                  |                      |                                                                       |                |        |          |

## INTEGRATION TEST CASE

In this, we should not write something which we already covered in the functional test cases, and something we have written in the integration test case should not be written in the system test case again.

Rules to write integration test cases

- Firstly, understand the product
- Identify the possible scenarios
- Write the test case based on the priority
- When the test engineer writing the test cases, they may need to consider the following aspects: If the test cases are in details:
  - They will try to achieve maximum test coverage.
  - All test case values or scenarios are correctly described.

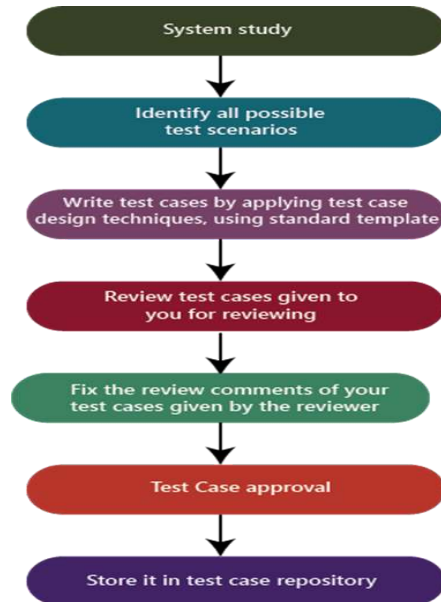
- They will try to think about the execution point of view.
- The template which is used to write the test case must be unique.

## SYSTEM TEST CASES

We will write the system test cases for the end-to-end business flows. And we have the entire modules ready to write the system test cases.

### The process to write test cases

The method of writing a test case can be completed into the following steps, which are as below:



#### System study

In this, we will understand the application by looking at the requirements or the SRS, which is given by the customer.

#### Identify all scenarios:

When the product is launched, what are the possible ways the end-user may use the software to identify all the possible ways.

I have documented all possible scenarios in a document, which is called test design/high-level design.

The test design is a record having all the possible scenarios.

#### Write test cases

Convert all the identified scenarios to test claims and group the scenarios related to their features, prioritize the module, and write test cases by applying test case design techniques and use the standard test case template, which means that the one which is decided for the project.

#### Review the test cases

Review the test case by giving it to the head of the team and, after that, fix the review feedback given by the reviewer.

#### Test case approval

After fixing the test case based on the feedback, send it again for the approval.

#### Store in the test case repository

After the approval of the particular test case, store in the familiar place that is known as the test case repository.

## BUG REPORTING

Bug reporting in software testing is a structured process of documenting and communicating issues found in software. It's a crucial part of ensuring the final product meets quality standards and functions as intended. Effective bug reports provide developers with the necessary information to understand, reproduce, and

fix issues, ultimately improving software reliability and stability.

### **Key Aspects of Bug Reporting:**

#### 1. Documentation:

Bug reports are written documents that detail software defects or issues.

#### 2. Communication:

They serve as a way for testers to communicate problems to developers and other stakeholders.

#### 3. Reproducibility:

Good bug reports include clear steps to reproduce the issue, allowing developers to easily verify the bug.

#### 4. Context:

They should provide relevant context, such as the test case, environment, and expected vs. actual outcomes.

#### 5. Severity and Priority:

Assessing the severity and priority of the bug helps prioritize fixing efforts.

#### 6. Visual Evidence:

Screenshots, videos, or logs can be included to visually demonstrate the issue and aid in understanding.

#### 7. Bug Tracking:

Bug reporting is often integrated with bug tracking systems to manage, track, and monitor the status of reported bugs.

### **Key Elements of a Good Bug Report:**

**Clear Title:** A concise and descriptive title summarizing the issue.

**Detailed Description:** A clear explanation of the issue, including when and how it occurred.

**Steps to Reproduce:** Step-by-step instructions on how to recreate the bug.

**Expected vs. Actual Results:** What was expected to happen versus what actually happened.

**Environment Details:** Information about the environment where the bug was found (e.g., browser, operating system).

**Visual Proof:** Screenshots, videos, or logs that demonstrate the bug.

**Severity and Priority:** An assessment of the bug's impact and urgency.

**Additional Information:** Relevant console logs, network requests, or source URLs can be included.

### **Benefits of Effective Bug Reporting:**

#### Improved Software Quality:

By identifying and addressing bugs early, the overall quality of the software is enhanced.

#### Faster Debugging:

Well-written bug reports allow developers to quickly understand and fix issues.

#### Streamlined Development Process:

Effective bug reporting facilitates better communication and collaboration between testers and developers.

#### Enhanced User Experience:

By fixing bugs, the user experience is improved, leading to increased user satisfaction.

## **SOFTWARE TESTING METRICS**

The purpose of software testing metrics is to increase the efficiency and effectiveness of the software testing process while also assisting in making better decisions for future testing by providing accurate data about the testing process.

### **Importance of Metrics in Software Testing:**

Test metrics are essential in determining the software's quality and performance. Developers may use the right software testing metrics to improve their

productivity.

**Early Problem Identification:** By measuring metrics such as defect density and defect arrival rate, testing teams can spot trends and patterns early in the development process.

**Allocation of Resources:** Metrics identify regions where testing efforts are most needed, which helps with resource allocation optimization. By ensuring that testing resources are concentrated on important areas, this enhances the strategy for testing as a whole.

**Monitoring Progress:** Metrics are useful instruments for monitoring the advancement of testing. They offer insight into the quantity of test cases that have been run, their completion rate, and if the testing effort is proceeding according to plan.

**Continuous Improvement:** Metrics offer input on the testing procedure, which helps to foster a culture of continuous development.

### **Types of Software Testing Metrics:**

Software testing metrics are divided into three categories:

**Process Metrics:** A project's characteristics and execution are defined by process metrics. These features are critical to the SDLC process's improvement and maintenance (Software Development Life Cycle).

**Product Metrics:** A product's size, design, performance, quality, and complexity are defined by product metrics. Developers can improve the quality of their software development by utilizing these features.

**Project Metrics:** Project Metrics are used to assess a project's overall quality. It is used to estimate a project's resources and deliverables, as well as to determine costs, productivity, and flaws.

### **Formula for Test Metrics:**

To get the percentage execution status of the test cases, the following formula can be used:

Percentage test cases executed = (No of test cases executed / Total no of test cases written) x 100

Similarly, it is possible to calculate for other parameters also such as test cases that were not executed, test cases that were passed, test cases that were failed, test cases that were blocked, and so on.

#### **1. Test Case Effectiveness:**

Test Case Effectiveness = (Number of defects detected / Number of test cases run) x 100

**2. Passed Test Cases Percentage:** Test Cases that Passed Coverage is a metric that indicates the percentage of test cases that pass.

Passed Test Cases Percentage = (Total number of tests ran / Total number of tests executed) x 100

**3. Failed Test Cases Percentage:** This metric measures the proportion of all failed test cases.

Failed Test Cases Percentage = (Total number of failed test cases / Total number of tests executed) x 100

**4. Blocked Test Cases Percentage:** During the software testing process, this parameter determines the percentage of test cases that are blocked.

Blocked Test Cases Percentage = (Total number of blocked tests / Total number of tests executed) x 100

**5. Fixed Defects Percentage:** Using this measure, the team may determine the percentage of defects that have been fixed.

Fixed Defects Percentage = (Total number of flaws fixed / Number of defects reported) x 100

**6. Rework Effort Ratio:** This measure helps to determine the rework effort ratio.

Rework Effort Ratio = (Actual rework efforts spent in that phase/ Total actual efforts spent in that phase) x 100

**7. Accepted Defects Percentage:** This measures the percentage of defects that are accepted out of the total accepted defects.

Accepted Defects Percentage = (Defects Accepted as Valid by Dev Team / Total



Defects Reported) x 100

8. Defects Deferred Percentage: This measures the percentage of the defects that are deferred for future release.

Defects Deferred Percentage = (Defects deferred for future releases / Total Defects Reported) x 100

**Example of Software Test Metrics Calculation:**

| S No. | Testing Metric                                           | Data retrieved during test case development |
|-------|----------------------------------------------------------|---------------------------------------------|
| 1     | No. of requirements                                      | 5                                           |
| 2     | The average number of test cases written per requirement | 40                                          |
| 3     | Total no. of Test cases written for all requirements     | 200                                         |
| 4     | Total no. of Test cases executed                         | 164                                         |
| 5     | No. of Test cases passed                                 | 100                                         |
| 6     | No. of Test cases failed                                 | 60                                          |
| 7     | No. of Test cases blocked                                | 4                                           |
| 8     | No. of Test cases unexecuted                             | 36                                          |
| 9     | Total no. of defects identified                          | 20                                          |
| 10    | Defects accepted as valid by the dev team                | 15                                          |
| 11    | Defects deferred for future releases                     | 5                                           |
| 12    | Defects fixed                                            | 12                                          |

**1. Percentage test cases executed** = (No of test cases executed / Total no of test cases written) x 100

= (164 / 200) x 100

= 82

**2. Test Case Effectiveness** = (Number of defects detected / Number of test cases run) x 100

= (20 / 164) x 100

= 12.2

**3. Failed Test Cases Percentage** = (Total number of failed test cases / Total number of tests executed) x 100

= (60 / 164) \* 100

= 36.59

**4. Blocked Test Cases Percentage** = (Total number of blocked tests / Total number of tests executed) x 100

= (4 / 164) \* 100

= 2.44

**5. Fixed Defects Percentage** = (Total number of flaws fixed / Number of defects reported) x 100

= (12 / 20) \* 100

= 60

**6. Accepted Defects Percentage** = (Defects Accepted as Valid by Dev Team / Total Defects Reported) x 100

= (15 / 20) \* 100

= 75

**7. Defects Deferred Percentage** = (Defects deferred for future releases / Total Defects Reported) x 100

= (5 / 20) \* 100

= 25